STEM Fusion

Introduction to Computational Thinking and Python Programming

FUNCTIONS AND DATA

Functions · Parameters & Arguments · Lists

Instructor: Sarom Leang, PhD
Specialist: Daniela Garcia Galindo
Innovation Hall

Pronouns

Instructor

Sarom Leang, Ph.D.

Pronouns: Teacher / Instructor

Specialist

Daniela Garcia Galindo

Pronouns: Daniela

Schedule

10:00 AM - 10:15 AM	Homeroom
10:15 AM - 11:35 AM	G1
11:40 AM - 12:35 PM	Lunch
12:40 PM - 02:00 PM	G2

Student Expectations

NO FOOD

NO DRINKS (on the table)

Be respectful to individuals and property

Be open to learning

Be open to not understanding

Be patient with yourself

Ask questions

Explore

Embrace failure

Course Overview

This course introduces the fundamental building blocks of computational thinking and computer programming using the Python language.

Upon successful completion of this course, students will be able to:

- Improve their problem-solving skills
- Write, read, and execute Python code using basic data types and operators

Course Overview (cont.)

Exploratory Topics

- ~~Session #1 – November 1, 2025~~
  - ~~Entrance Survey~~
  - ~~How Computers Work~~
- ~~Session #2 – December 6, 2025~~
  - ~~Parallel Computing~~
- ~~Session #3 – January 31, 2025~~
  - ~~Generative Artificial Intelligence~~
- Session #4 – March 7, 2026
  - Generative Artificial Intelligence
- Session #5 – April 11, 2026
  - Future of Computing (Quantum)
  - Exit survey

Topics

- Algorithms and Problem Solving
- Debugging and Troubleshooting
- Loops
- Algorithms and Syntax
- Variables and Conditionals
- Variable Arithmetic
- Conditionals (If/Else)
- Compound Conditionals

What Are We Doing Today?

1	2	3
		
Functions	Parameters & Arguments	Lists (Arrays)

Functions

Write once, use everywhere — the Netflix model of code

Why Do We Need Functions? 🤔

❌ Without functions — repetitive

```
# Greeting for Alex
print('--- Welcome ---')
print('Hello, Alex!')
print('Glad you are here.')
print('---------------')

# Greeting for Jordan
print('--- Welcome ---')
print('Hello, Jordan!')
print('Glad you are here.')
print('---------------')

# Greeting for Sam...
# Copy paste again?
```

✅ With a function — clean & reusable

```
def greet(name):
    print('--- Welcome ---')
    print('Hello,', name + '!')
    print('Glad you are here.')
    print('---------------')

# Call it 3 times:
greet('Alex')
greet('Jordan')
greet('Sam')

# Change format in ONE place.
```

DRY — Don't Repeat Yourself. One of the most important rules in programming.

Defining and Calling a Function

Anatomy of a Function Definition

```
def greet():           # function name
    print('Hello!')    # function body (runs when called)
```

1 DEFINE

Write the function once using def. The body is indented. This is the recipe.

2 CALL

Use the function name followed by () to run it. This executes the body.

3 REUSE

Call it as many times as you like. The body runs each time.

Return Values — Getting an Answer Back

print only — output vanishes

```
def add(a, b):
    print(a + b)

result = add(3, 4)
# result is None!
```

return — value you can use ✅

```
def add(a, b):
    return a + b

result = add(3, 4)
print(result)  # 7
```

Return value in action

```
def get_grade(score):
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    else:
        return 'F'

g = get_grade(85)
print('Your grade:', g)
# Your grade: B
```

Parameters & Arguments

Sending data into a function — the inputs that make it flexible

Parameters vs Arguments 

Parameters are the placeholders. Arguments are the actual values passed in.

PARAMETER — in the definition

```
def greet(name):     # name is
                     # a PARAMETER
    print('Hi,', name)
```

ARGUMENT — in the call

```
greet('Alex')    # 'Alex' is
                 # an ARGUMENT
# Output: Hi, Alex
```

PARAMETER

In the def line

→

maps to

ARGUMENT

In the call

Multiple Parameters 

Multiple parameters — Uber fare

```
def calculate_fare(miles, rate, surge):
    base = miles * rate
    total = base * surge
    return round(total, 2)

# Short ride, no surge
fare1 = calculate_fare(2, 1.50, 1.0)
print('Fare:', fare1)  # 3.0

# Long ride, surge pricing
fare2 = calculate_fare(8, 1.50, 2.5)
print('Fare:', fare2)  # 30.0
```

⚠ Order matters!

```
calculate_fare(1.5, 2, 1.0)
# Wrong! miles=1.5, rate=2
# Order must match definition
```

Keyword arguments (bonus)

You can pass arguments by name to avoid order confusion:

```
calculate_fare(
  surge=2.5, miles=8,
  rate=1.50)
# Order no longer matters!
```

Default Parameter Values 🎯

Syntax — default after =

```
def func(required,
optional='default'):
    # optional has fallback value
    pass
```

The rule:

Parameters with default values must always come after parameters without defaults.

Example: welcome message 👏

```
def welcome(name,
            greeting='Hello',
            lang='English'):
    print(greeting + ',', name)
    print('Language:', lang)

# Use all defaults:
welcome('Alex')
# Hello, Alex  /  Language: English

# Override one default:
welcome('Maria', greeting='Hola')
# Hola, Maria  /  Language: English

# Override both:
welcome('Jean', 'Bonjour', 'French')
# Bonjour, Jean  /  Language: French
```

⚡ OPTIONAL CHALLENGE — Parameters & Arguments

Pen & Paper

On paper, write a function called make_playlist_entry that:
- Takes: title (required), artist (required), plays=0 (default)
- Returns a formatted string like:
  '♪ Anti-Hero by Taylor Swift (plays: 150)'

Then write 3 calls:
1. With plays provided
2. Without plays (use default)
3. Using keyword arguments

Trace each call — what does each one return?

Try it in IDLE

```
def make_playlist_entry(
        title, artist, plays=0):
    return ('♪ ' + title +
            ' by ' + artist +
            ' (plays: ' + str(plays) + ')')

# 1. With plays
print(make_playlist_entry(
    'Anti-Hero', 'Taylor Swift', 150))

# 2. Without plays
print(make_playlist_entry(
    'Flowers', 'Miley Cyrus'))

# 3. Keyword args
print(make_playlist_entry(
    plays=99,
    artist='SZA',
    title='Kill Bill'))
```

Work individually • There's no wrong answer — just think it through!

ANSWERS — Parameters & Arguments

Pen & Paper

On paper, write a function called make_playlist_entry that:
- Takes: title (required), artist (required), plays=0 (default)
- Returns a formatted string like:
  '♪ Anti-Hero by Taylor Swift (plays: 150)'

Then write 3 calls:
1. With plays provided
2. Without plays (use default)
3. Using keyword arguments

Trace each call — what does each one return?

IDLE Output

```
>>> print(make_playlist_entry(
...     'Anti-Hero', 'Taylor Swift', 150))
♪ Anti-Hero by Taylor Swift (plays: 150)

>>> print(make_playlist_entry(
...     'Flowers', 'Miley Cyrus'))
♪ Flowers by Miley Cyrus (plays: 0)

>>> print(make_playlist_entry(
...     plays=99,
...     artist='SZA',
...     title='Kill Bill'))
♪ Kill Bill by SZA (plays: 99)

# Common mistake:
# print('... (plays: ' + plays + ')')
# TypeError — must use str(plays)
# int can't concatenate with str
```

Lists (Arrays)

Store, organize, and process collections of data

Creating Lists and Indexing

A list stores multiple values in order, inside square brackets [ ], separated by commas.

Creating a list

```
playlist = ['Anti-Hero',
            'Flowers',
            'As It Was']

scores = [88, 92, 74, 96]

mixed = ['Alex', 21, True]
```

Indexing — accessing items

```
songs = ['Anti-Hero', 'Flowers',
         'As It Was']

print(songs[0])   # Anti-Hero
print(songs[1])   # Flowers
print(songs[2])   # As It Was
print(songs[-1])  # As It Was (last)
print(len(songs)) # 3
```

index [0]	index [1]	index [2]
"Anti-Hero"	"Flowers"	"As It Was"

Common List Methods 

.append(x)

Add x to end of list

```
songs.append('Kill Bill')
# Songs now has 4 items
```

.remove(x)

Remove first x found

```
songs.remove('Flowers')
# Flowers is gone
```

len(list)

Count items in list

```
count = len(songs)
print(count)  # 3
```

.sort()

Sort list in place

```
scores.sort()
# [74, 88, 92, 96]
```

.pop(i)

Remove & return item

```
last = songs.pop()
# removes & returns last
```

x in list

Check if x is inside

```
'Flowers' in songs
# True or False
```

Lists and Loops — The Power Pair

Process every item — grade report

```
def letter_grade(score):
    if score >= 90: return 'A'
    elif score >= 80: return 'B'
    elif score >= 70: return 'C'
    else: return 'F'

scores = [85, 92, 67, 78, 95]

for score in scores:
    grade = letter_grade(score)
    print(score, '->', grade)

# 85 -> B
# 92 -> A
# 67 -> F  ...
```

Build a filtered list — top scores only

```
scores = [85, 92, 67, 78, 95, 88]

honor_roll = []

for score in scores:
    if score >= 88:
        honor_roll.append(score)

print('Honor roll:', honor_roll)
# Honor roll: [92, 95, 88]

print('Count:', len(honor_roll))
# Count: 3
```

⚡ OPTIONAL CHALLENGE — Lists (Arrays)

Pen & Paper

Write a function called top_songs that:
• Takes a list of song dictionaries (with 'title' and 'plays' keys)
• Returns a NEW list of titles where plays > 100

Example input:
songs = [
 {'title': 'Anti-Hero', 'plays': 150},
 {'title': 'Flowers', 'plays': 80},
 {'title': 'Kill Bill', 'plays': 120}
]

Expected output: ['Anti-Hero', 'Kill Bill']

Bonus: sort the result alphabetically.

Try it in IDLE

```
# Simpler version first:
songs = [150, 80, 120, 200, 55]
def top_plays(plays_list, threshold=100):
    hits = []
    for p in plays_list:
        if p > threshold:
            hits.append(p)
    return hits

print(top_plays(songs))
# [150, 120, 200]

# Challenge version with dicts:
song_data = [ {'title': 'Anti-Hero', 'plays': 150},
              {'title': 'Flowers',   'plays': 80},
              {'title': 'Kill Bill', 'plays': 120} ]
hits = [s['title'] for s in song_data
        if s['plays'] > 100]
print(hits)
# ['Anti-Hero', 'Kill Bill']
```

Work individually • There's no wrong answer — just think it through!

ANSWERS — Lists (Arrays)

Pen & Paper

Write a function called top_songs that:
• Takes a list of song dictionaries (with 'title' and 'plays' keys)
• Returns a NEW list of titles where plays > 100

Example input:
songs = [
 {'title': 'Anti-Hero', 'plays': 150},
 {'title': 'Flowers', 'plays': 80},
 {'title': 'Kill Bill', 'plays': 120}
]

Expected output: ['Anti-Hero', 'Kill Bill']

Bonus: sort the result alphabetically.

IDLE Output

```
# Simpler version first:
songs = [150, 80, 120, 200, 55]
def top_plays(plays_list, threshold=100):
    hits = []
    for p in plays_list:
        if p > threshold:
            hits.append(p)
    return hits

print(top_plays(songs))
# [150, 120, 200]

# Challenge version with dicts:
song_data = [ {'title': 'Anti-Hero', 'plays': 150},
              {'title': 'Flowers',   'plays': 80},
              {'title': 'Kill Bill', 'plays': 120} ]
hits = [s['title'] for s in song_data
        if s['plays'] > 100]
print(hits)
# ['Anti-Hero', 'Kill Bill']
```

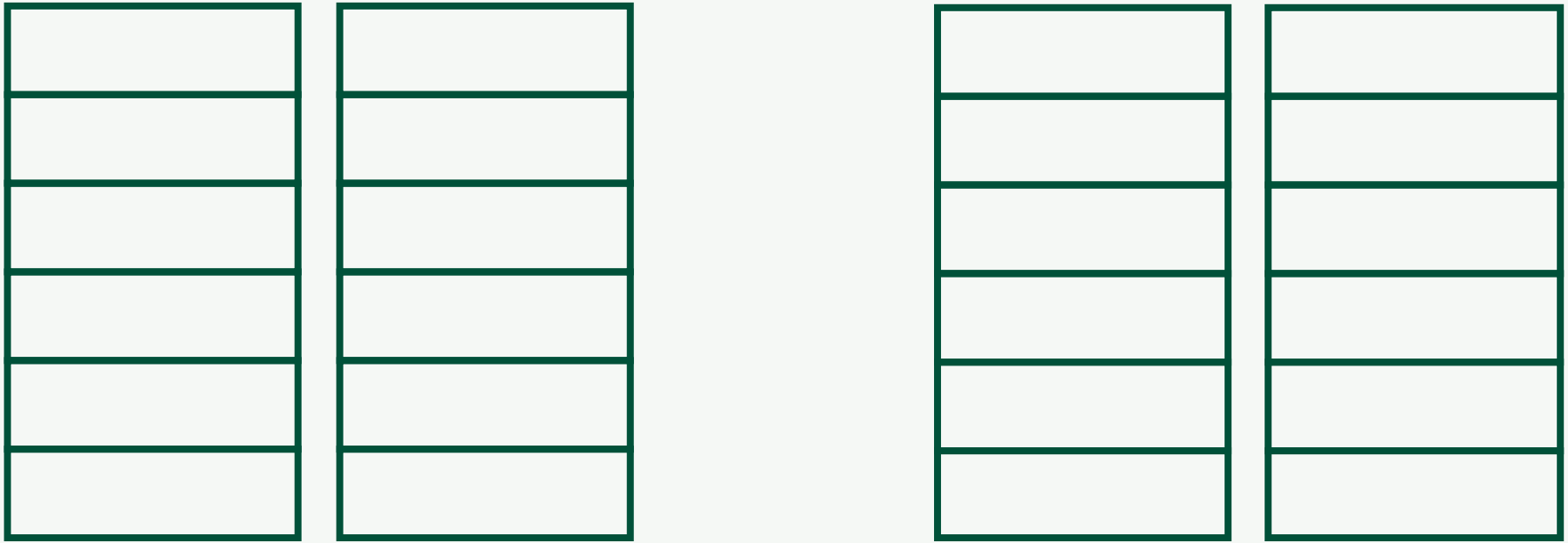
Complete!

Three sessions. One complete foundation in Python. Look what you can do now:

- Algorithms & pseudocode
- Python syntax & variables
- Arithmetic & strings
- Boolean logic
- If / elif / else
- For & while loops
- Functions & parameters
- Lists & iteration

What's next? Object-Oriented Programming · APIs · Web Development · Data Science · Game Dev

Innovation Computer Issues



Front Screen